Variational autoencoders

A generator with fewer random inputs than pixels has no probability density over images. How can one still be trained by maximum likelihood?

Nov 2, 2026 Retrieved Nov 7, 2026

You will learn

- Why a network that compresses and rebuilds images does not produce new ones when we feed it random numbers.
- How a little noise gives a generator a probability for every image, and why that probability is still out of reach.
- How a lower bound on that probability lets us train anyway, and how far it falls short on handwritten digits.
- How the training loss splits into two terms that pull against each other, and how a gradient passes through a random draw.
- What the trained model does with its inputs, where it fails and why its samples come out blurry.

Before you start

- The pushforward, the manifold hypothesis and why generators have no density (Chapter 1)
- Gibbs' inequality, maximum likelihood as KL, upper and lower surrogates and the two directions of KL (Chapter 2)
- Multivariate Gaussians and their covariance matrices, for the linear model

A network can be trained to copy an image from its input to its output. Copying sounds pointless until we force the image through a narrow middle layer, only a few numbers wide. The first half of the network then has to shrink the image to those few numbers, and the second half has to rebuild the image from them.

Look at the second half on its own. It takes a few numbers and returns an image, which is what a generator does. So perhaps we already have a generator, and all it needs is random numbers as input.

We can try this. We train such a network on handwritten digits with 8 numbers in the middle, draw 8 random numbers $1{,}000$ times and pass them to the second half. A digit classifier recognizes only 3 of the resulting images.

The same network rebuilds digits it has never seen, so its second half can produce digits when it receives the right numbers. Random numbers are rarely the right ones. This chapter is about how to train the network so that they are, which leads to the *variational autoencoder* (VAE) (Kingma and Welling 2014; Rezende, Mohamed, and Wierstra 2014).

Autoencoders

The network of the opening is called an *autoencoder*. The figure below shows one trained on handwritten digits, copying its input to its output through a narrow middle layer.

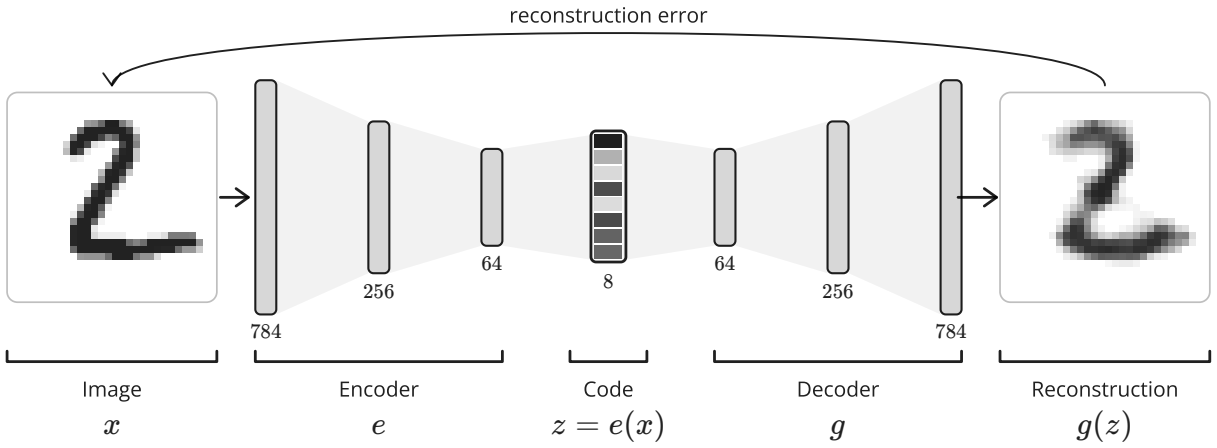


Figure 1. An autoencoder trained on handwritten digits. The encoder e squeezes the 784 pixels of the image through layers of 256 and 64 units down to a code of only 8 numbers (shaded cells). The decoder g mirrors it and rebuilds the image from the code alone. The input digit comes from the test set, unseen during training, and the reconstruction is the network's real output. The reconstruction keeps the digit and loses its finest details.

Definition 1 (Autoencoder)**.** An *autoencoder* is a pair of maps. The *encoder* $e : \mathbb{R}^d \to \mathbb{R}^k$ turns an image x into a *code* $z = e(x)$, and the *decoder* $g : \mathbb{R}^k \to \mathbb{R}^d$ turns a code back into an image $\widehat{x} = g(e(x))$, with $k < d$. Given training images $X_1, \dots, X_n$, both are trained to minimize the *reconstruction error*

$$\frac{1}{n} \sum_{i=1}^{n} \left\| g(e(X_i)) - X_i \right\|^2.$$

A code as long as the image could hold every pixel, and the networks could simply pass it through. A code of 8 numbers cannot, so training pushes the encoder to keep the information that most reduces the reconstruction error, and the decoder rebuilds the image from what was kept.

What the autoencoder learns depends on how flexible e and g are, and the two extremes show the range. If both are affine maps (linear plus a constant), the best autoencoder projects the data onto the span of its first k principal components (Baldi and Hornik 1989), so a nonlinear autoencoder generalizes principal component analysis. If both are very flexible, they can reach zero error on the training set by memorizing it, with the encoder sending the i-th training image to the number i and the decoder sending i back to the i-th image. This is the copying model of Chapter 1 again.

It may seem odd that 8 numbers can describe a 784-pixel digit, but this is what Chapter 1 would lead us to expect. Digits concentrate near a manifold (Chapter 1, Assumption 5) whose dimension Pope et al. (2021) estimate between 7 and 13. The decoder *parametrizes* that manifold, since $g(z)$ moves along it as z moves, and the encoder gives *coordinates* that locate an image on it. A code of 8 numbers sits at the low end of that range, so it keeps the main factors of variation and loses the rest, as the reconstruction above shows.

From autoencoder to generator

If the decoder moves along the surface where digits live, the obvious way to generate a digit is to pick a code at random and decode it. We draw the code from $\mathcal{N}(0, I)$, the distribution γ of Chapter 1, which from now on we call the *prior*. Codes drawn this way almost never give a digit, as the measurements at the end of this section show.

The trouble is that the decoder was trained only on the n codes of the training images, and nothing in the training controls where these codes lie. A code drawn from the prior may fall far from all of them and decode into a blend of digits or into nothing recognizable. Fitting a single Gaussian to the codes after training does not solve this, because the Gaussian still puts mass between the codes, where the decoder is untrained.

So a decoder that generates needs two properties of the code space that the autoencoder never asked for. Small moves of the code should give small changes of the image, including between the codes of training images, where a plain autoencoder's decoder is unconstrained. Every code that the prior may produce should also decode into a plausible image. The VAE works towards both with two changes to the autoencoder, and the first replaces each code by a small cloud of codes.

Definition 2 (Gaussian encoder)**.** A *Gaussian encoder* is a neural network that maps an image x to a mean $\mu(x) \in \mathbb{R}^k$ and standard deviations $\sigma(x) \in (0, +\infty)^k$. It defines the distribution of codes

$$q(z \mid x) = \mathcal{N}\big(\mu(x), \operatorname{diag}(\sigma(x)^2)\big).$$

During training the code of x is drawn from $q(z \mid x)$, and the decoder must rebuild x from whichever code comes out, so every code near the centre of the cloud must decode into something close to x. That gives us the first property inside one cloud. The clouds also connect the images to each other. Where the clouds of two images overlap, the decoder must produce something between the two images, so it is trained on the space between training codes as well.

Clouds alone are not enough, because the encoder could shrink $\sigma(x)$ to zero and fall back to a plain autoencoder, or push the clouds far apart so that they never overlap. The second change is aimed at the second property. It adds to the loss a term that grows when $q(z \mid x)$ moves away

from $\mathcal{N}(0, I)$, the KL divergence from the cloud to the prior, whose formula we derive later. The reconstruction term favours small and well-separated clouds, while the penalty favours clouds that look like the prior. At the balance between the two terms, the clouds move into the region where the prior puts its mass while still telling images apart. With a code of 2 numbers they nearly fill that region. With 8 numbers there is far more room to fill, and we return to this when the model fails.

To see what each term does, we train the same architecture three ways. If the two changes work, the codes of the VAE should spread over the disk of the prior, and random codes should decode into digits.

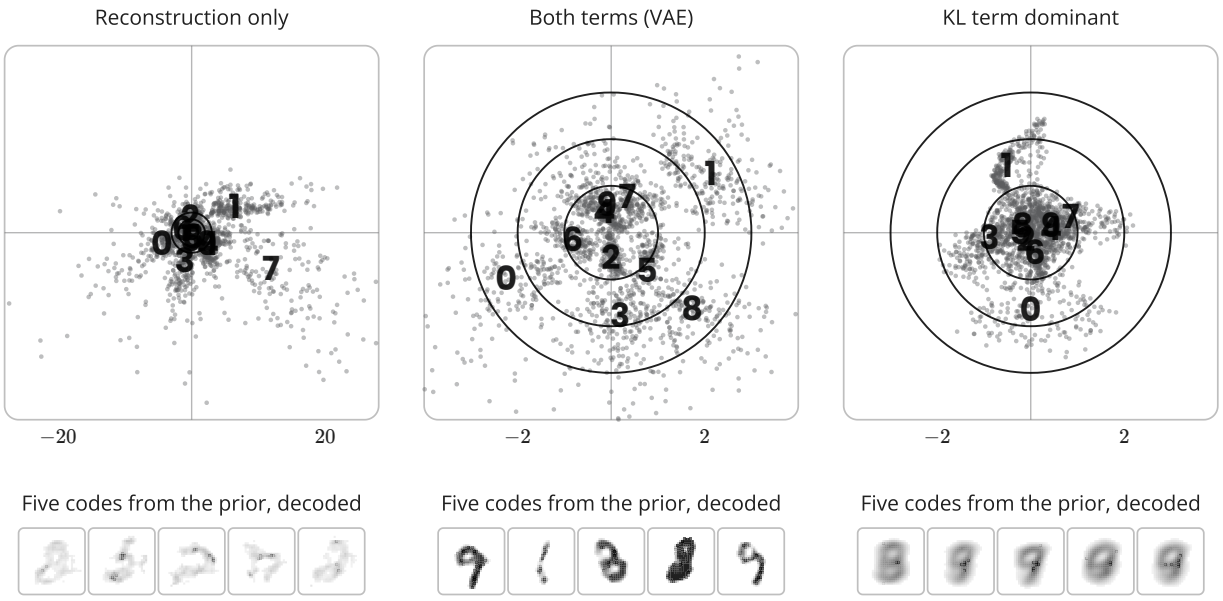


Figure 2. A plain autoencoder (reconstruction only), a VAE and a VAE whose KL term is weighted 100 times more, with the same architecture. The top row uses codes of $k = 2$ numbers and shows the encoder means of $2{,}000$ test digits, the circles of radius 1, 2 and 3 of the prior and the median code of each class (the left panel's axis is seven times wider). The bottom row decodes the same five codes from $\mathcal{N}(0, I_8)$ with the $k = 8$ version of each model.

With reconstruction only, the codes spread far beyond the prior. The prior's disk covers only the crowded centre where classes overlap, and prior codes decode into faint blends. With both terms the classes spread over the prior's disk and the same five codes decode into digits, although the count below shows that many other codes do not. When the penalty dominates, most coordinates of the code collapse onto the prior. The code then carries almost no information, and prior codes decode into a few nearly identical blurred shapes.

The figure shows only five samples, so we count. If nearly every prior code decoded into a clear digit, the share recognized would come close to that of real digits.

Lab

What the two terms do, measured

We decode $1{,}000$ codes drawn from $\mathcal{N}(0, I_8)$ with each 8-dimensional model. A fixed digit classifier with 98.9% test accuracy, used throughout the course, says whether it recognizes a digit with more than 90% confidence.

Images	Recognized as a digit
plain autoencoder, prior codes	0.3%
VAE, prior codes	46.7%
VAE whose penalty dominates, prior codes	15.1%
real test digits (all 10,000)	96.8%

The codes of the plain autoencoder with $k = 2$ have standard deviations of 5.8 and 4.3 where the prior has 1. Of these codes, 95% lie within radius 16.7, against 2.45 for the prior. The balanced VAE's clouds have an average standard deviation of 0.05 and a KL to the prior of 24.8 nats per image. In the model whose penalty dominates, 6 of the 8 coordinates of the code no longer vary across images, the clouds have an average standard deviation of 0.92 and their KL to the prior is 0.83 nats. Retraining the classifier moved the recognized shares by up to about one percentage point in two retrainings. Each model was trained once, with one seed.

The two changes lift the share from 0.3% to 46.7%, about half of what real digits reach, and a heavier penalty brings it back down to 15.1%. So the balance matters. We chose both changes because they looked sensible, and nothing so far tells us why this penalty is the right one or how much of it to use. The next section starts from a different problem, and its solution answers both questions.

A generator with a density

Chapter 2 ended with an obstacle. Maximum likelihood needs a density, and a generator with fewer latent dimensions than pixels has none. Chapter 1 sketched the repair, which is to add a little Gaussian noise to the output, and we now ask what that lets us compute. We start from the simplest generator we can draw, a line in the plane with noise around it. With $k = 1$ and $d = 2$, take a vector $a \in \mathbb{R}^2$ and a point $b \in \mathbb{R}^2$ and draw

$$X = aZ + b + \sigma_{\text{dec}}\,\varepsilon, \qquad Z \sim \mathcal{N}(0,1), \quad \varepsilon \sim \mathcal{N}(0, I_2),$$

with Z and ε independent. Without the noise every sample lies on the line $\{az + b : z \in \mathbb{R}\}$, which has zero area, and the distribution has no density. With the noise the samples form a band around the line. The point X is then Gaussian with mean b and covariance $aa^\top + \sigma_{\text{dec}}^2 I_2$, the sum of the covariances of aZ and of the noise, and its density is positive everywhere in the plane.

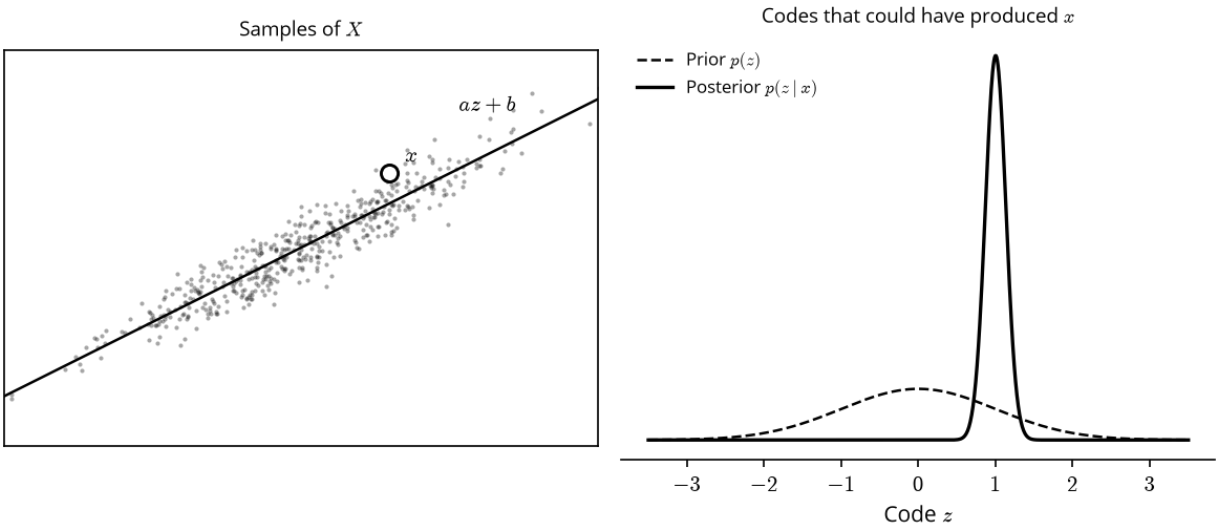


Figure 3. The linear model with $a = (1, 0.5)$, $b = 0$ and $\sigma_{\text{dec}} = 0.15$. Left, 500 samples of X around the line $az + b$, and one point x (ring). Right, as functions of the code z, the prior $\mathcal{N}(0, 1)$ (dashed) and the posterior of the codes that could have produced x (solid). The posterior is centred where the line passes closest to x and is 7.5 times narrower than the prior.

The noise also lets us ask something new of the model, which is the set of codes that could have produced a given point x. The right panel shows them, and this is the object the rest of the chapter turns on. Codes whose point $az + b$ lies near x are likely, so they form a narrow Gaussian over z when the noise is small.

The same construction works for any generator. A *Gaussian decoder* with variance $\sigma^2_{\text{dec}} > 0$ turns a code z into the distribution $\mathcal{N}\big(g_\theta(z), \sigma^2_{\text{dec}} I_d\big)$, centred on the output of a network $g_\theta : \mathbb{R}^k \to \mathbb{R}^d$ with weights θ. From now on we write $p(z)$ for the density of the prior and $p_\theta(x \mid z)$ for the density of this Gaussian. The model draws Z from the prior and then X from $p_\theta(\cdot \mid Z)$. To find how likely an image is, we go through every code, ask how likely that code is to produce the image and weight the answer by how likely the code is, so X has the density

$$p_\theta(x) = \int_{\mathbb{R}^k} p_\theta(x \mid z)\, p(z)\, dz,$$

a mixture of Gaussians with one component for every code.

This density is what maximum likelihood was missing. It lies between 0 and $(2\pi\sigma^2_{\text{dec}})^{-d/2}$ at every image, and the log-likelihood has a finite average over the data when the images and the decoder outputs have finite second moments (Proposition 3). Chapter 2's result on maximum likelihood asks for exactly this finite average, so maximizing the average log-likelihood of the noisy model minimizes an estimate of $\mathrm{KL}(\mathbb{P}^* \,\|\, \mathbb{P}_\theta)$ up to a constant.

Going further

Why the noisy model has a log-likelihood

Proposition 3 (The noisy model has a log-likelihood). *For every $x \in \mathbb{R}^d$, $0 < p_\theta(x) \leq (2\pi\sigma_{\mathrm{dec}}^2)^{-d/2}$ and*

$$\log p_\theta(x) \geq -\frac{d}{2}\log\left(2\pi\sigma_{\mathrm{dec}}^2\right) - \frac{1}{2\sigma_{\mathrm{dec}}^2}\,\mathbb{E}_{Z\sim p(z)}\big\|x - g_\theta(Z)\big\|^2.$$

If $\mathbb{E}\|X\|^2$ is finite for $X \sim \mathbb{P}^*$ and $\mathbb{E}\|g_\theta(Z)\|^2$ is finite for $Z \sim p(z)$, then $\mathbb{E}_{X\sim\mathbb{P}^*}|\log p_\theta(X)|$ is finite.

Proof.

The Gaussian density $p_\theta(x \mid z) = (2\pi\sigma_{\mathrm{dec}}^2)^{-d/2}\exp\big(-\|x - g_\theta(z)\|^2/(2\sigma_{\mathrm{dec}}^2)\big)$ lies in $\big(0, (2\pi\sigma_{\mathrm{dec}}^2)^{-d/2}\big]$ for every z. Integrating against the prior, whose total mass is 1, keeps $p_\theta(x)$ in the same interval. For the lower bound on the logarithm, write $p_\theta(x) = \mathbb{E}_{Z\sim p(z)}[p_\theta(x \mid Z)]$ and apply Jensen's inequality to the concave logarithm,

$$\log p_\theta(x) \geq \mathbb{E}_{Z\sim p(z)}\big[\log p_\theta(x \mid Z)\big] = -\frac{d}{2}\log\left(2\pi\sigma_{\mathrm{dec}}^2\right) - \frac{1}{2\sigma_{\mathrm{dec}}^2}\,\mathbb{E}_{Z\sim p(z)}\big\|x - g_\theta(Z)\big\|^2.$$

So $\log p_\theta(x)$ lies between this lower bound and the constant $-\frac{d}{2}\log(2\pi\sigma_{\mathrm{dec}}^2)$, and

$$\big|\log p_\theta(x)\big| \leq \frac{d}{2}\big|\log\left(2\pi\sigma_{\mathrm{dec}}^2\right)\big| + \frac{1}{2\sigma_{\mathrm{dec}}^2}\,\mathbb{E}_{Z}\big\|x - g_\theta(Z)\big\|^2.$$

By $\|x - y\|^2 \leq 2\|x\|^2 + 2\|y\|^2$, the expectation is at most $2\|x\|^2 + 2\,\mathbb{E}\|g_\theta(Z)\|^2$. Taking the expectation over $X \sim \mathbb{P}^*$ gives a finite bound under the two moment assumptions. □

In one case we can compute everything, and we keep that case throughout the chapter to check what happens where we cannot. When the decoder is linear, the density and the codes that explain an image have closed forms. The model with $g(z) = Az + b$ for a matrix $A \in \mathbb{R}^{d\times k}$ and a vector $b \in \mathbb{R}^d$ is *probabilistic PCA* (Tipping and Bishop 1999). It is the probabilistic version of the linear autoencoder of the first section.

Proposition 4 (The linear model). *Let $Z \sim \mathcal{N}(0, I_k)$ and $p(x \mid z) = \mathcal{N}(Az + b, \sigma_{\mathrm{dec}}^2 I_d)$, and write $M = A^\top A + \sigma_{\mathrm{dec}}^2 I_k$. Then*

$$p(x) = \mathcal{N}\big(x;\, b,\, AA^\top + \sigma_{\mathrm{dec}}^2 I_d\big), \qquad p(z \mid x) = \mathcal{N}\big(z;\, M^{-1}A^\top(x - b),\, \sigma_{\mathrm{dec}}^2 M^{-1}\big).$$

Proof.

The model writes $X = AZ + b + \sigma_{\mathrm{dec}}\varepsilon$ with $Z \sim \mathcal{N}(0, I_k)$ and $\varepsilon \sim \mathcal{N}(0, I_d)$ independent. An affine map of a Gaussian vector is Gaussian, so $AZ + b$ is Gaussian with mean b and covariance $AI_kA^\top = AA^\top$. The sum of two independent Gaussian vectors is Gaussian, with the sum of the

means and the sum of the covariances, which gives $p(x)$.

For the posterior, Bayes' rule gives $p(z \mid x) = p(x \mid z)\,p(z)/p(x)$, and $p(x)$ does not depend on z. Collect the terms of $\log p(x \mid z) + \log p(z)$ that depend on z and write c for any term that does not,

$$\begin{aligned}
\log p(z \mid x) &= -\frac{1}{2\sigma_{\text{dec}}^2}\|x - b - Az\|^2 - \frac{1}{2}\|z\|^2 + c \\
&= -\frac{1}{2\sigma_{\text{dec}}^2}\Big(z^\top A^\top A z - 2\,z^\top A^\top (x - b)\Big) - \frac{1}{2\sigma_{\text{dec}}^2}\,\sigma_{\text{dec}}^2\, z^\top z + c \\
&= -\frac{1}{2\sigma_{\text{dec}}^2}\Big(z^\top M z - 2\,z^\top A^\top (x - b)\Big) + c.
\end{aligned}$$

The matrix M is symmetric and positive definite, since $z^\top M z = \|Az\|^2 + \sigma_{\text{dec}}^2\|z\|^2 > 0$ for $z \neq 0$, so it is invertible. With $\bar{z} = M^{-1}A^\top(x - b)$ we have $A^\top(x - b) = M\bar{z}$, and completing the square gives

$$z^\top M z - 2\,z^\top M \bar{z} = (z - \bar{z})^\top M (z - \bar{z}) - \bar{z}^\top M \bar{z},$$

where $\bar{z}^\top M \bar{z}$ does not depend on z. Hence $\log p(z \mid x) = -\frac{1}{2}(z - \bar{z})^\top \big(\sigma_{\text{dec}}^2 M^{-1}\big)^{-1}(z - \bar{z}) + c$, which is the logarithm of the Gaussian density with mean $\bar{z}$ and covariance $\sigma_{\text{dec}}^2 M^{-1}$, up to a constant. Two densities whose logarithms differ by a constant are equal, since both integrate to 1. □

In the plane the posterior mean $a^\top(x - b)/(\|a\|^2 + \sigma_{\text{dec}}^2)$ is the code of the projection of x on the line, shrunk slightly towards 0. The posterior variance $\sigma_{\text{dec}}^2/(\|a\|^2 + \sigma_{\text{dec}}^2)$ is 0.018 in the figure. So the formula says what the picture suggested, that the likely codes sit around the closest point of the line.

With a neural decoder neither formula exists. The density $p_\theta(x)$ is an integral of a network over $\mathbb{R}^k$, and the posterior $p_\theta(z \mid x) = p_\theta(x \mid z)\,p(z)/p_\theta(x)$ has it in its denominator. Most of what follows is about getting around this, and it helps to have the standard names that Bayesian statistics gives the pieces of the model.

Network or quantity	Probabilistic object	Name
the fixed distribution of codes, $\mathcal{N}(0, I)$	$p(z)$	prior
the decoder plus noise	$p_\theta(x \mid z)$	conditional likelihood
the distribution of images the model defines	$p_\theta(x)$	likelihood, or evidence
the codes that could have produced x	$p_\theta(z \mid x)$	posterior
the encoder	$q_\phi(z \mid x)$	approximate posterior

The Gaussian encoder of the last section gets weights ϕ as a subscript. The table already calls it an approximate posterior, and the next section shows why. The density $p_\theta(x)$ is called the *evidence* because it measures how well the model as a whole explains the observed image.

We should flag one simplification before going on. The images we display are decoder means $g_\theta(Z)$. On MNIST we use $\sigma^2_{\text{dec}} = 0.03$, a noise of standard deviation 0.17 on pixels in $[0, 1]$ that would make samples speckled. The guarantees of the chapter concern the noisy model, and the displayed images come from a different distribution that the likelihood does not score.

The evidence lower bound

Maximum likelihood asks for $\log p_\theta(X_i)$ for every training image, and each one is an integral over $\mathbb{R}^k$. We reach a usable target in three steps. We try the plainest estimate of the integral and see it fail. We improve it with the encoder. We then turn it into a lower bound that can be estimated and differentiated from one code per image, and whose gap to the log-likelihood has an exact form.

Why sampling the prior fails

The integral is an average over the prior, so the plainest estimate draws codes from the prior and averages $p_\theta(x \mid z)$. This estimates $p_\theta(x)$ without bias, and on MNIST it is useless. For each of 100 test digits we drew $100{,}000$ codes from the prior and decoded them with the 8-dimensional VAE of this chapter. For the median digit a single code carried essentially all of the weight, and the estimate of $\log p_\theta(x)$ was 85 nats below the one obtained with the encoder in the next subsection. I expected prior sampling to be noisy. I did not expect one code out of $100{,}000$ to carry nearly all the weight.

The region of good codes is tiny, and the Gaussian decoder punishes everything outside it. With $\sigma^2_{\text{dec}} = 0.03$, two decoded images whose squared errors to x differ by 3 differ in $p_\theta(x \mid z)$ by a factor $\exp(3/0.06) = \exp(50)$, and the codes that decode close to a given digit fill a tiny part of the prior's mass.

The linear model shows how this scales, since there the number of prior codes needed for a given precision can be computed exactly (Proposition 5), and it grows geometrically with the latent dimension. With $k = 8$ latent coordinates and the noise level of our VAE, a relative error of 10% needs at least 9×10^6 prior codes for a single image, and every coordinate added multiplies this by at least 4.2.

Going further

How many prior codes the linear model needs

The relative variance of the estimate from S prior codes is $\big(\mathbb{E}[\hat{p}^2]/p(x)^2 - 1\big)/S$, where $\hat{p} = p(x \mid Z)$ is the estimate from one code, so the second moment below sets the number of codes needed.

> **Proposition 5** (Prior sampling in the linear model). *In the linear model let A have the entries $A_{jj} = a > 0$ for $j \le k$ and 0 elsewhere, so that each latent coordinate moves one coordinate of the image. Write $r = a^2/\sigma_{\mathrm{dec}}^2$. For every $x \in \mathbb{R}^d$ the estimate $\hat{p} = p(x \mid Z)$ with $Z \sim \mathcal{N}(0, I_k)$ satisfies*
>
> $$\frac{\mathbb{E}\big[\hat{p}^{\,2}\big]}{p(x)^2} = \prod_{j=1}^{k} \frac{1+r}{\sqrt{1+2r}}\, \exp\Big(\frac{r\,(x_j - b_j)^2}{\sigma_{\mathrm{dec}}^2 (1+r)(1+2r)}\Big) \;\ge\; \Big(\frac{1+r}{\sqrt{1+2r}}\Big)^{k},$$
>
> *with equality at $x = b$. The factor $(1+r)/\sqrt{1+2r}$ is larger than 1 for every $r > 0$.*

Proof.

Write $y_j = x_j - b_j$. The density $p(x \mid z)$ factorizes over the coordinates of x. For $j > k$ the factor $\mathcal{N}(x_j; b_j, \sigma_{\mathrm{dec}}^2)$ does not depend on z, and by Proposition 4 the same factor appears in $p(x)$, whose covariance $AA^\top + \sigma_{\mathrm{dec}}^2 I_d$ is diagonal with entries $a^2 + \sigma_{\mathrm{dec}}^2$ for $j \le k$ and σ_{dec}^2 for $j > k$. These factors cancel in the ratio. The coordinates $Z_1, \dots, Z_k$ are independent, so the expectation of the product of the remaining factors is the product of their expectations, and it suffices to treat one coordinate $j \le k$.

The square of a Gaussian density is a rescaled Gaussian density with half the variance,

$$\mathcal{N}(y; \mu, \sigma_{\mathrm{dec}}^2)^2 = \frac{1}{2\pi\sigma_{\mathrm{dec}}^2} \exp\Big(-\frac{(y-\mu)^2}{\sigma_{\mathrm{dec}}^2}\Big) = \frac{1}{2\sqrt{\pi}\,\sigma_{\mathrm{dec}}}\, \mathcal{N}\big(y; \mu, \sigma_{\mathrm{dec}}^2/2\big),$$

since $\mathcal{N}(y; \mu, \sigma_{\mathrm{dec}}^2/2) = (\pi\sigma_{\mathrm{dec}}^2)^{-1/2} \exp(-(y-\mu)^2/\sigma_{\mathrm{dec}}^2)$. Averaging $\mathcal{N}(y; aZ_j, \sigma_{\mathrm{dec}}^2/2)$ over $Z_j \sim \mathcal{N}(0,1)$ gives the density at y of $aZ_j + (\sigma_{\mathrm{dec}}/\sqrt{2})\,\varepsilon$, which is $\mathcal{N}(y; 0, a^2 + \sigma_{\mathrm{dec}}^2/2)$. Hence

$$\mathbb{E}\big[\mathcal{N}(y_j; aZ_j, \sigma_{\mathrm{dec}}^2)^2\big] = \frac{1}{2\sqrt{\pi}\,\sigma_{\mathrm{dec}}} \cdot \frac{1}{\sqrt{\pi(2a^2 + \sigma_{\mathrm{dec}}^2)}} \exp\Big(-\frac{y_j^2}{2a^2 + \sigma_{\mathrm{dec}}^2}\Big).$$

The square of the j-th factor of $p(x)$ is $\mathcal{N}(y_j; 0, a^2 + \sigma_{\mathrm{dec}}^2)^2 = \big(2\pi(a^2 + \sigma_{\mathrm{dec}}^2)\big)^{-1} \exp\big(-y_j^2/(a^2 + \sigma_{\mathrm{dec}}^2)\big)$. Dividing,

$$\frac{\mathbb{E}\big[\mathcal{N}(y_j; aZ_j, \sigma_{\mathrm{dec}}^2)^2\big]}{\mathcal{N}(y_j; 0, a^2+\sigma_{\mathrm{dec}}^2)^2} = \frac{a^2+\sigma_{\mathrm{dec}}^2}{\sigma_{\mathrm{dec}}\sqrt{2a^2+\sigma_{\mathrm{dec}}^2}} \exp\Big(y_j^2\Big(\frac{1}{a^2+\sigma_{\mathrm{dec}}^2} - \frac{1}{2a^2+\sigma_{\mathrm{dec}}^2}\Big)\Big) = \frac{1+r}{\sqrt{1+2r}} \exp\Big(\frac{}{\sigma_{\mathrm{dec}}^2(}$$

where the last step divides numerator and denominator by σ_{dec}^2 and uses $\frac{1}{a^2+\sigma_{\mathrm{dec}}^2} - \frac{1}{2a^2+\sigma_{\mathrm{dec}}^2} = \frac{a^2}{(a^2+\sigma_{\mathrm{dec}}^2)(2a^2+\sigma_{\mathrm{dec}}^2)}$. The exponent is non-negative and vanishes at $y_j = 0$, which gives the inequality and the case of equality. Finally $(1+r)^2 = 1 + 2r + r^2 > 1 + 2r$ for $r > 0$. □

The factor grows like $\sqrt{r/2}$ when the noise is small. With $a = 1$ and $\sigma_{\mathrm{dec}}^2 = 0.03$ it is 4.2, and with $k = 8$ the second moment is at least 9.2×10^4 times $p(x)^2$. A relative error of 10% then needs at least 9×10^6 prior codes for a single image, and every latent coordinate added multiplies this number by at least 4.2. For a typical image of the model $(x_j - b_j)^2$ is about $a^2 + \sigma_{\mathrm{dec}}^2$, and the factor of each coordinate rises to about 6.8.

Importance sampling with the encoder

Prior sampling fails because almost every code it draws is a bad one for this image. The natural repair is to draw codes from a distribution that favours the good ones, and the linear model shows how far this can go. There the posterior is known, and drawing the codes from it makes every sample give the exact answer.

Drawing from a different distribution changes the average, so we need a correction. This is *importance sampling*. It estimates an expectation under one distribution with samples from another distribution called the *proposal*, and it corrects each sample by a weight. With a proposal $q(z)$ that is positive wherever $p(z)$ is, the weight $w(z) = p_\theta(x \mid z)\, p(z)/q(z)$ satisfies

$$p_\theta(x) = \int q(z)\, \frac{p_\theta(x \mid z)\, p(z)}{q(z)}\, dz = \mathbb{E}_{z\sim q}\big[w(z)\big].$$

The ratio $p(z)/q(z)$ makes the correction. A code that the proposal draws more often than the prior would is scaled down, and one it draws less often is scaled up. With the prior itself as proposal the ratio is 1, and we are back to the estimate of the last subsection. Each proposal gives the right average, so what separates them is how much the weights vary.

> **Proposition 6** (The posterior is the best proposal). *For every proposal q positive wherever $p(z)$ is, the weight $w(Z)$ with $Z \sim q$ has mean $p_\theta(x)$. Its variance is zero if and only if q is the posterior $p_\theta(z \mid x)$, and then $w(z) = p_\theta(x)$ for every z.*

Proof.

The mean is the identity above. Bayes' rule gives $p_\theta(x \mid z)\, p(z) = p_\theta(z \mid x)\, p_\theta(x)$, so $w(z) = p_\theta(x)\, p_\theta(z \mid x)/q(z)$. If q is the posterior, $w(z) = p_\theta(x)$ for every z and the variance is zero. Conversely, a variance of zero means that $w(Z)$ equals its mean $p_\theta(x)$ with probability 1. Then $p_\theta(z \mid x) = q(z)$ for q-almost every z, and since both are densities that integrate to 1, they are equal as distributions. □

So the ideal proposal is the posterior, the one distribution we cannot write down for a neural decoder. We do have something close at hand. The Gaussian encoder already produces a cloud of codes for each image, and that cloud can serve as the proposal. If it is trained to come close to $p_\theta(z \mid x)$, which the bound below will arrange, it deserves the name *approximate posterior* $q_\phi(z \mid x)$.

On MNIST it is a much better proposal than the prior. The 85 nats of the last subsection compare $1{,}000$ codes from the encoder of each test digit with $100{,}000$ codes from the prior, for the median digit. The weights still vary a lot. Their *effective number* $(\sum_i w_i)^2 / \sum_i w_i^2$ is 2.4 out of $1{,}000$ for the median digit, so only a few codes count.

The bound

We now have a way to estimate the likelihood of an image. We draw codes from the encoder, compute the weight $w(z)$ of each one and average the weights. For training this is still not what we need. Training maximizes the logarithm of the likelihood, so the quantity to estimate is $\log \mathbb{E}[w]$, the logarithm of an average. The average of a few weights is an unbiased estimate of $\mathbb{E}[w]$. Its logarithm is a biased estimate of $\log \mathbb{E}[w]$, and its gradient is biased too.

To make progress we swap the two operations. We take the logarithm of each weight first and average afterwards. With a single code this gives $\log w(z)$, which we can compute and differentiate directly. The swap changes the value, and we can say in which direction. The logarithm is concave, so Jensen's inequality gives $\log \mathbb{E}[w] \geq \mathbb{E}[\log w]$. The new quantity therefore never exceeds the log-likelihood, which makes it a lower bound. A lower bound is a useful training target only if it stays close to what it bounds, so we also need the size of the gap. The theorem gives both.

> **Theorem 7** (Evidence lower bound). *For every image x and every density $q_\phi(z \mid x)$,*
>
> $$\log p_\theta(x) \geq \mathcal{L}_{\theta,\phi}(x) = \mathbb{E}_{z \sim q_\phi(z \mid x)} \left[\log \frac{p_\theta(x \mid z)\, p(z)}{q_\phi(z \mid x)} \right].$$
>
> *The gap is exactly a KL divergence, $\log p_\theta(x) - \mathcal{L}_{\theta,\phi}(x) = \mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p_\theta(z \mid x)\big)$. The quantity $\mathcal{L}_{\theta,\phi}(x)$ is the evidence lower bound (ELBO).*

Proof.

Bayes' rule gives $p_\theta(x \mid z)\, p(z) = p_\theta(z \mid x)\, p_\theta(x)$ for every z. Substituting it inside the logarithm,

$$\mathcal{L}_{\theta,\phi}(x) = \mathbb{E}_q \left[\log p_\theta(x) + \log \frac{p_\theta(z \mid x)}{q_\phi(z \mid x)} \right] = \log p_\theta(x) - \mathbb{E}_q \left[\log \frac{q_\phi(z \mid x)}{p_\theta(z \mid x)} \right] = \log p_\theta(x) - \mathrm{KL}\big(q_\phi(z \mid$$

where the first term leaves the expectation because it does not depend on z. By Gibbs' inequality the gap is non-negative, and it vanishes exactly when the encoder equals the true posterior. □

Take the gap formula first. Its left side is how far the bound falls short for this image, and its right side says the shortfall is the KL from the encoder's cloud to the true posterior, so the bound is exact when the two coincide and loose when the encoder misses. Two extreme encoders show the range. If q_ϕ is the posterior, every weight equals $p_\theta(x)$ by Proposition 6, the swap of logarithm and average changes nothing and the gap is 0. If q_ϕ is the prior for every image, the ELBO becomes $\mathbb{E}_{z\sim p(z)}[\log p_\theta(x \mid z)]$, and the gap is the KL from the wide prior to a posterior that the figure of the linear model shows can be far narrower.

The ELBO is the expected logarithm of the importance weight, $\mathcal{L}_{\theta,\phi}(x) = \mathbb{E}_q[\log w]$, while the log-likelihood is the logarithm of its expectation. Unlike the log-likelihood it contains only things we can evaluate, namely the decoder, the prior and the encoder. The posterior that we could not compute appears only in the gap, which we never have to evaluate.

The VAE maximizes the ELBO jointly in θ and ϕ. The log-likelihood does not depend on ϕ, so raising the ELBO in ϕ can only close the gap. This is what trains the encoder to approach the posterior. Raising it in θ can raise the likelihood or close the gap, and we return to this below.

What the bound costs

The theorem speaks about one image, while training averages over all of them. To compare that average with the distance of Chapter 2, it helps to look at an image and its code together, as a pair. There are two ways to produce such a pair, and the average of the negative bound turns out to compare them. Up to the constant $\mathbb{E}\log p^*(X)$, it is a KL divergence between the two, and this gives its exact relation to the distance. The data and the encoder define one distribution of pairs, which draws an image $X \sim \mathbb{P}^*$ and then a code from $q_\phi(\cdot \mid X)$. The model defines another, which draws a code from the prior and then an image from the decoder.

> **Proposition 8** (The negative ELBO is a joint KL)**.** *Let $\mathbb{P}^*$ have a density p^* with $\mathbb{E}|\log p^*(X)| < \infty$, let $q_\phi(x,z) = p^*(x)\,q_\phi(z \mid x)$ and $p_\theta(x,z) = p(z)\,p_\theta(x \mid z)$, and assume the expectations below are finite. Then*
>
> $$\begin{aligned}\mathrm{KL}\big(q_\phi(x,z) \,\|\, p_\theta(x,z)\big) &= -\,\mathbb{E}_{X\sim\mathbb{P}^*}\big[\mathcal{L}_{\theta,\phi}(X)\big] + \mathbb{E}_{X\sim\mathbb{P}^*}\big[\log p^*(X)\big] \\ &= \mathrm{KL}\big(\mathbb{P}^* \,\|\, \mathbb{P}_\theta\big) + \mathbb{E}_{X\sim\mathbb{P}^*}\,\mathrm{KL}\big(q_\phi(\cdot \mid X) \,\|\, p_\theta(\cdot \mid X)\big).\end{aligned}$$

Proof.

By definition, $\mathrm{KL}(q_\phi(x,z) \,\|\, p_\theta(x,z)) = \mathbb{E}_{(X,Z)\sim q_\phi}\big[\log q_\phi(X,Z) - \log p_\theta(X,Z)\big]$. Split $\log q_\phi(X,Z) = \log p^*(X) + \log q_\phi(Z \mid X)$ and take the expectation over Z first, for a fixed X,

$$\mathrm{KL}(q_\phi(x,z) \,\|\, p_\theta(x,z)) = \mathbb{E}_X\big[\log p^*(X)\big] + \mathbb{E}_X\,\mathbb{E}_{Z\sim q_\phi(\cdot\mid X)}\left[\log \frac{q_\phi(Z \mid X)}{p(Z)\,p_\theta(X \mid Z)}\right].$$

The inner expectation is $-\mathcal{L}_{\theta,\phi}(X)$, which gives the first equality. For the second, Theorem 7 gives for each X

$$-\mathcal{L}_{\theta,\phi}(X) = -\log p_\theta(X) + \mathrm{KL}\big(q_\phi(\cdot \mid X) \,\|\, p_\theta(\cdot \mid X)\big),$$

and $\mathbb{E}[\log p^*(X)] - \mathbb{E}[\log p_\theta(X)] = \mathrm{KL}(\mathbb{P}^* \,\|\, \mathbb{P}_\theta)$ by Chapter 2, Proposition 9. □

Read the second line of the proposition from left to right. For the two distributions of pairs to agree, the images have to agree and so do the codes of each image. The first term measures the images and the second measures the codes. The proposition assumes that the data have a density, which is compatible with Chapter 1, since data that concentrate near a manifold can still have one.

In the language of Chapters 1 and 2, the distance of the VAE is $R(\theta) = \mathrm{KL}(\mathbb{P}^* \,\|\, \mathbb{P}_\theta)$, the direction of maximum likelihood. Its surrogate is the averaged negative ELBO, which equals the joint KL above up to the constant $\mathbb{E}\log p^*(X)$ and is estimated on the training images. The gap is the average KL from the encoder to the posterior, and it is never negative, so the surrogate is an *upper bound* on the distance. This is the safe side of Chapter 2, where driving the surrogate down also drives the distance down.

We can now write what the bound costs. Give each decoder its best encoder and write $\mathrm{gap}(\theta)$ for the average KL that remains. Let $\hat{\theta}$ be the decoder that minimizes the surrogate. Then Chapter 2's proposition on upper and lower surrogates gives $R(\hat{\theta}) \le R(\theta) + \mathrm{gap}(\theta)$ for every θ, so the trained decoder is as good as any other decoder up to that decoder's gap. Optimizing ϕ only shrinks the gap. Optimizing θ can also shrink it by making the posterior easier to approximate, so the bound favours decoders whose posteriors look like diagonal Gaussians (Cremer, Li, and Duvenaud 2018).

The gap can also be measured, at least from below. We estimate $\log p_\theta(x)$ by the logarithm of the average weight of $1{,}000$ encoder codes. The logarithm of an average falls below $\log p_\theta(x)$ on average by Jensen's inequality, as the ELBO does. The difference between this estimate and the ELBO is therefore a lower bound on the gap, up to sampling noise. For the median test digit it is about 14 nats, and the ELBO of that digit is about 468.

A positive log-likelihood may look wrong, but a density can exceed 1. Here the Gaussian decoder contributes the constant $-\frac{d}{2}\log(2\pi\sigma_{\mathrm{dec}}^2) \approx +654$ nats. The average ELBO over the test set is lower than the median, about $654 - 212 - 25 \approx 417$ nats, where we subtract the two parts of the loss that the next section separates.

In the linear model the gap can vanish, because the posterior of Proposition 4 has a mean linear in x and a fixed covariance, which a linear Gaussian encoder can match (a diagonal one when $A^\top A$ is diagonal).

Going further

Three KL divergences, and the name of the method

Three KL divergences appear in a VAE. Maximum likelihood minimizes $\mathrm{KL}(\mathbb{P}^* \,\|\, \mathbb{P}_\theta)$ in image space, whose forward direction pushes the model to cover every mode of the data. The gap $\mathrm{KL}(q_\phi \,\|\, p_\theta(z \mid x))$ lives in code space in the mode-seeking direction of Chapter 2's lab, so a

diagonal Gaussian encoder tends to underestimate the spread of a correlated posterior and to pick one mode of a posterior with several. The penalty $\mathrm{KL}(q_\phi \,\|\, p(z))$ of the next section pulls each cloud towards the prior.

The name of the method comes from the encoder. Finding the Gaussian (μ, σ) that maximizes the ELBO of one image is an optimization over a family of distributions, which is what *variational inference* means. The encoder *amortizes* this work, since one network answers for every image in a single forward pass. The gap therefore has two parts (Cremer, Li, and Duvenaud 2018, sec. 3.1). The *approximation gap* exists because the best diagonal Gaussian is not the posterior, and the *amortization gap* because one network does not find the best Gaussian for every image.

In the linear model the ELBO has no local maxima that the likelihood lacks (Lucas et al. 2019).

Reconstruction and regularization

The bound is still one expectation with a ratio inside it. To train on it, and to compare it with what we did by hand earlier, we split it. The split is one line.

Proposition 9 (The two terms of the ELBO)**.**

$$\mathcal{L}_{\theta,\phi}(x) = \mathbb{E}_{z\sim q_\phi(z\mid x)}\big[\log p_\theta(x \mid z)\big] - \mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z)\big).$$

Proof.

Split the logarithm in the definition of $\mathcal{L}_{\theta,\phi}(x)$ into $\log p_\theta(x \mid z) + \log\big(p(z)/q_\phi(z \mid x)\big)$. The expectation of the second part under $q_\phi(z \mid x)$ is $-\mathrm{KL}(q_\phi(z \mid x) \,\|\, p(z))$ by the definition of KL. □

These are the reconstruction term and the penalty of the section “From autoencoder to generator”. We added them there because they looked sensible, and they now come out of the likelihood with nothing added. On MNIST the two have very different sizes. On the test set the KL from the encoder to the prior averages 24.8 nats per image, and the squared reconstruction error averages 12.7, which contributes $12.7/(2 \times 0.03) \approx 212$ nats.

That division by $2\sigma_{\mathrm{dec}}^2$ comes from the Gaussian decoder, whose log-density is

$$\log p_\theta(x \mid z) = -\frac{1}{2\sigma_{\mathrm{dec}}^2}\,\|x - g_\theta(z)\|^2 - \frac{d}{2}\log\big(2\pi\sigma_{\mathrm{dec}}^2\big).$$

With σ_{dec}^2 held fixed, the first term of the ELBO is therefore the reconstruction error of an autoencoder multiplied by $-1/(2\sigma_{\mathrm{dec}}^2)$, up to a constant. The second term has a closed form, which is the exact penalty we postponed.

Property 10 (KL between a Gaussian and the standard Gaussian)**.** For $\mu \in \mathbb{R}^k$ and $\sigma \in (0, +\infty)^k$,

$$\mathrm{KL}\big(\mathcal{N}(\mu, \mathrm{diag}(\sigma^2)) \,\|\, \mathcal{N}(0, I)\big) = \frac{1}{2} \sum_{j=1}^{k} \big(\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2\big).$$

Proof.

Both densities are products over the coordinates, so the logarithm of their ratio is a sum over coordinates and the KL is the sum of the KL divergences of the coordinates. For one coordinate, with $Z \sim \mathcal{N}(\mu_j, \sigma_j^2)$,

$$\log \frac{\mathcal{N}(Z; \mu_j, \sigma_j^2)}{\mathcal{N}(Z; 0, 1)} = \log \frac{(2\pi\sigma_j^2)^{-1/2} \exp\big(-(Z-\mu_j)^2/(2\sigma_j^2)\big)}{(2\pi)^{-1/2} \exp\big(-Z^2/2\big)} = -\frac{1}{2}\log \sigma_j^2 - \frac{(Z-\mu_j)^2}{2\sigma_j^2} + \frac{Z^2}{2}.$$

Take the expectation term by term. $\mathbb{E}[(Z-\mu_j)^2] = \sigma_j^2$, so the middle term has expectation $-\frac{1}{2}$. $\mathbb{E}[Z^2] = \mathrm{Var}(Z) + (\mathbb{E}Z)^2 = \sigma_j^2 + \mu_j^2$, so the last term has expectation $\frac{1}{2}(\sigma_j^2 + \mu_j^2)$. The sum is $\frac{1}{2}(\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2)$. □

Each term vanishes exactly when $\mu_j = 0$ and $\sigma_j = 1$. The penalty grows when a cloud drifts from the origin and when it shrinks or spreads compared with the prior. Dropping constants and averaging over the training images, the VAE minimizes

$$\frac{1}{n} \sum_{i=1}^{n} \left(\frac{1}{2\sigma_{\mathrm{dec}}^2} \, \mathbb{E}_{z \sim q_\phi(z|X_i)} \big\| X_i - g_\theta(z) \big\|^2 + \frac{1}{2} \sum_{j=1}^{k} \big(\mu_j(X_i)^2 + \sigma_j(X_i)^2 - 1 - \log \sigma_j(X_i)^2\big) \right).$$

This is an autoencoder loss with a noisy code and a penalty on the code distribution. The section “From autoencoder to generator” left open how much penalty to use. The formula now fixes the weight, which is set by the decoder variance. A small σ_{dec}^2 favours precise reconstructions and a large one favours codes close to the prior. The β-VAE instead multiplies the KL term by a factor $\beta > 0$ (Higgins et al. 2017), and the next proposition shows that this amounts to changing the decoder variance.

Proposition 11 (The weight of the KL and the decoder variance)**.** *For every $\beta > 0$, the loss above with the KL term multiplied by β has the same minimizers (θ, ϕ) as the loss with σ_{dec}^2 replaced by $\beta\sigma_{\mathrm{dec}}^2$ and no factor on the KL term.*

Proof.

Multiplying a function by the positive constant $1/\beta$ does not change its minimizers. Applied to $\frac{1}{2\sigma_{\mathrm{dec}}^2}\mathbb{E}\|X - g_\theta(Z)\|^2 + \beta\,\mathrm{KL}$, it gives $\frac{1}{2\beta\sigma_{\mathrm{dec}}^2}\mathbb{E}\|X - g_\theta(Z)\|^2 + \mathrm{KL}$, the loss with variance $\beta\sigma_{\mathrm{dec}}^2$. □

So the weight on the KL and the decoder variance are one setting under two names. The likelihood can also choose this setting for us. For a single variance shared by all pixels the best value is the mean squared error per pixel, which is 0.016 on our 8-dimensional VAE, about half of the 0.03 we trained with. Training with that value changes little, as the fold-out below shows.

Going further

The best decoder variance

Proposition 12 (The best decoder variance)**.** *For a fixed encoder and a fixed decoder mean g_θ, the variance σ^2_{dec} that maximizes the average ELBO is the mean squared error per pixel,*

$$\sigma^2_{\text{dec}} = \frac{1}{d}\,\mathbb{E}\left\|X - g_\theta(Z)\right\|^2, \qquad X \sim \mathbb{P}^*,\ Z \sim q_\phi(\cdot \mid X).$$

Proof.

Write $v = \sigma^2_{\text{dec}}$ and $E = \mathbb{E}\|X - g_\theta(Z)\|^2$. By Proposition 9 and the Gaussian log-density, the part of the average ELBO that depends on v is $h(v) = -E/(2v) - \frac{d}{2}\log(2\pi v)$, since the KL term does not involve v. Its derivative is $h'(v) = E/(2v^2) - d/(2v) = (E - dv)/(2v^2)$, positive for $v < E/d$ and negative for $v > E/d$. So h is maximized at $v = E/d$. □

We can check how far our fixed choice is from this value. Our 8-dimensional VAE, trained with $\sigma^2_{\text{dec}} = 0.03$, reaches a mean squared error of 0.016 per pixel on the test set. By Proposition 11 it behaves like a model with the best variance and $\beta = 0.03/0.0162 \approx 1.85$. A VAE of the same architecture trained with the variance set to its best value at every step ends at 0.016 as well. Its squared error per image is 12.4 against 12.7, its KL to the prior 28.0 nats against 24.8, and 44.0% of its prior samples are recognized against 46.7%. Each model was trained once, with one seed, so gaps of this size may be run-to-run noise. Over this factor of 1.85 the choice of variance changes little.

Training

Gradients of an expectation

Both terms of the loss are expectations over codes drawn from the encoder, and training needs their gradients. For the decoder's weights this causes no difficulty, since they do not affect how the codes are drawn. The encoder's weights do. Changing them changes the distribution the codes are drawn from, so the gradient cannot simply move inside the expectation. A general estimator exists for this case, the *score-function* (or REINFORCE) estimator. Its variance is high, almost four times that of the estimator below on the simplest example (Proposition 14).

The VAE takes another route, which is to separate the randomness from the weights. We draw fixed noise first and then let the encoder shift and scale it, so the code becomes a function of the noise.

Lemma 13 (Reparameterization). *Let $\varepsilon \sim \mathcal{N}(0, I_k)$ and $z = \mu_\phi(x) + \sigma_\phi(x) \odot \varepsilon$, where $\odot$ is the coordinate-wise product. Then $z \sim q_\phi(z \mid x)$, and for every function u whose expectations exist*

$$\mathbb{E}_{z \sim q_\phi(z|x)}[u(z)] = \mathbb{E}_{\varepsilon \sim \mathcal{N}(0,I)}\big[u\big(\mu_\phi(x) + \sigma_\phi(x) \odot \varepsilon\big)\big].$$

If u is differentiable and $\phi \mapsto u(\mu_\phi(x) + \sigma_\phi(x) \odot \varepsilon)$ has a gradient bounded by an integrable function of ε near each ϕ, the gradient in ϕ of the right-hand side is the expectation of the gradient of the integrand.

Proof.

Each coordinate $z_j = \mu_j + \sigma_j \varepsilon_j$ is an affine function of a standard Gaussian, so it is Gaussian with mean μ_j and variance σ_j^2. The coordinates are independent because the ε_j are. This is $q_\phi(z \mid x)$, and the identity is the expectation of $u(z)$ computed through the variable ε that produces z. The exchange of gradient and expectation is the standard theorem of differentiation under the integral sign, which the domination assumption allows (dominated convergence). □

On the right-hand side the distribution of ε does not depend on ϕ, and ϕ only enters through a differentiable function. Under the conditions of the lemma the gradient can now move inside, and one draw of ε gives an unbiased estimate of it. The code is now the pushforward of a fixed random variable by a map that depends on the parameters, as in Chapter 1.

Going further

Two gradient estimators

The difference between the two estimators already appears in one dimension. Take $Z \sim \mathcal{N}(\mu, \sigma^2)$ and $u(z) = z^2$, so that $\mathbb{E}[u(Z)] = \mu^2 + \sigma^2$ and its derivative in μ is 2μ. Two estimates of this derivative can be computed from one draw of Z.

The first, the *score-function* (or REINFORCE) estimator, works for any distribution with a differentiable density q_ϕ. Differentiating under the integral and writing $\nabla_\phi q_\phi = q_\phi \nabla_\phi \log q_\phi$,

$$\nabla_\phi \, \mathbb{E}_{z \sim q_\phi}[u(z)] = \int u(z) \, \nabla_\phi q_\phi(z) \, dz = \mathbb{E}_{z \sim q_\phi}\big[u(z) \, \nabla_\phi \log q_\phi(z)\big],$$

so $u(Z) \, \nabla_\phi \log q_\phi(Z)$ is an unbiased estimate. For the Gaussian, $\partial_\mu \log q = (z - \mu)/\sigma^2$. The second estimator writes $Z = \mu + \sigma\varepsilon$ with fixed noise $\varepsilon \sim \mathcal{N}(0, 1)$ and differentiates through this function, which gives $\partial_\mu u(\mu + \sigma\varepsilon) = 2(\mu + \sigma\varepsilon)$.

Proposition 14 (Variance of the two estimators). *For $Z \sim \mathcal{N}(0, \sigma^2)$ and $u(z) = z^2$, both estimators of the derivative in μ at $\mu = 0$ have mean 0. The pathwise estimate $2Z$ has variance $4\sigma^2$, and the score-function estimate Z^3/σ^2 has variance $15\sigma^2$.*

Proof.

At $\mu = 0$ the pathwise estimate is $2(0 + \sigma\varepsilon) = 2Z$ and the score-function estimate is $Z^2 \cdot Z/\sigma^2 = Z^3/\sigma^2$. Both have mean 0 because the odd moments of a centred Gaussian vanish. The variances are $\mathbb{E}[4Z^2] = 4\sigma^2$ and $\mathbb{E}[Z^6]/\sigma^4 = 15\sigma^6/\sigma^4 = 15\sigma^2$, using $\mathbb{E}[Z^6] = 15\sigma^6$ (the sixth moment of a centred Gaussian is $5 \cdot 3 \cdot 1\,\sigma^6$). □

The score-function estimate ignores the shape of u and only weights its values, and its variance is almost four times larger in this simplest case. The VAE uses the pathwise estimate, which needs a continuous code and a differentiable decoder.

One code per image

A VAE draws one code per image at every step. Given how much the importance weights varied, this may seem too few, yet on MNIST it is enough for the gradient. For the median test digit, the logarithm of the importance weight of $1{,}000$ encoder codes has a standard deviation of 6.7 nats. The weights themselves vary by a factor $\exp(6.7) \approx 800$ between codes one standard deviation apart, which is why their effective number was 2.4. Working with the logarithm of the weights keeps the estimate stable. The single-code estimates of the ELBO and of its gradient are also unbiased, so their errors have no preferred direction, and they average out over the 128 images of a minibatch and over the steps.

The training step

One training step proceeds as follows.

1. The encoder reads a minibatch and outputs $\mu_\phi(x)$ and $\log \sigma_\phi(x)^2$ (the logarithm keeps σ positive without a constraint).
2. The step draws $\varepsilon \sim \mathcal{N}(0, I_k)$ for each image and forms $z = \mu_\phi(x) + \sigma_\phi(x) \odot \varepsilon$.
3. The decoder maps z to $g_\theta(z)$, and the loss adds the squared error divided by $2\sigma_{\text{dec}}^2$ to the closed-form KL of Property 10.
4. Backpropagation runs through the decoder, the code and the encoder, and an optimizer updates θ and ϕ.

In PyTorch the loss reads

```
def vae_loss(x, encoder, decoder, var_dec):
    mu, logvar = encoder(x)
    z = mu + torch.exp(0.5 * logvar) * torch.randn_like(mu)   # reparameterization
    reconstruction = ((decoder(z) - x) ** 2).sum(dim=1) / (2 * var_dec)
    kl = 0.5 * (mu ** 2 + logvar.exp() - 1 - logvar).sum(dim=1)
    return (reconstruction + kl).mean()
```

The main VAEs of this chapter have $k = 2$ or $k = 8$. Their encoders have layers of sizes $784 \to 512 \to 256 \to 2$ and $784 \to 256 \to 64 \to 8$, and their decoders mirror them. They were trained on MNIST with this loss and $\sigma^2_{\text{dec}} = 0.03$, using Adam with learning rate 10^{-3} on batches of 128 for 30 epochs. Other settings are given where they are used.

In practice

A learned decoder variance needs the term $\frac{d}{2} \log \sigma^2_{\text{dec}}$ that the loss above drops, otherwise it grows without limit. A single shared variance then settles at the value of Proposition 12 (Rybkin, Daniilidis, and Levine 2021). A variance per pixel and per code is riskier, because a decoder that passes through one training image with a variance shrinking to zero there sends the likelihood to infinity (Mattei and Frellsen 2018).

Going further

Two more training tips

Binary cross-entropy, common as the reconstruction term on MNIST, is not a normalized density for grey pixels and biases the model. The continuous Bernoulli distribution corrects it (Loaiza-Ganem and Cunningham 2019).

Two tricks keep the optimizer from switching the code off before the decoder has learned to use it, which matters most with expressive decoders. One raises the weight of the KL term from 0 to 1 over the first epochs (Bowman et al. 2016). The other leaves the KL of each group of latent coordinates unpenalized below an allowance of a few nats (Kingma et al. 2016).

The latent space

We now have a trained model and can look at what it learned. With a code of $k = 2$ numbers we can see the decoder's output over the whole latent plane.

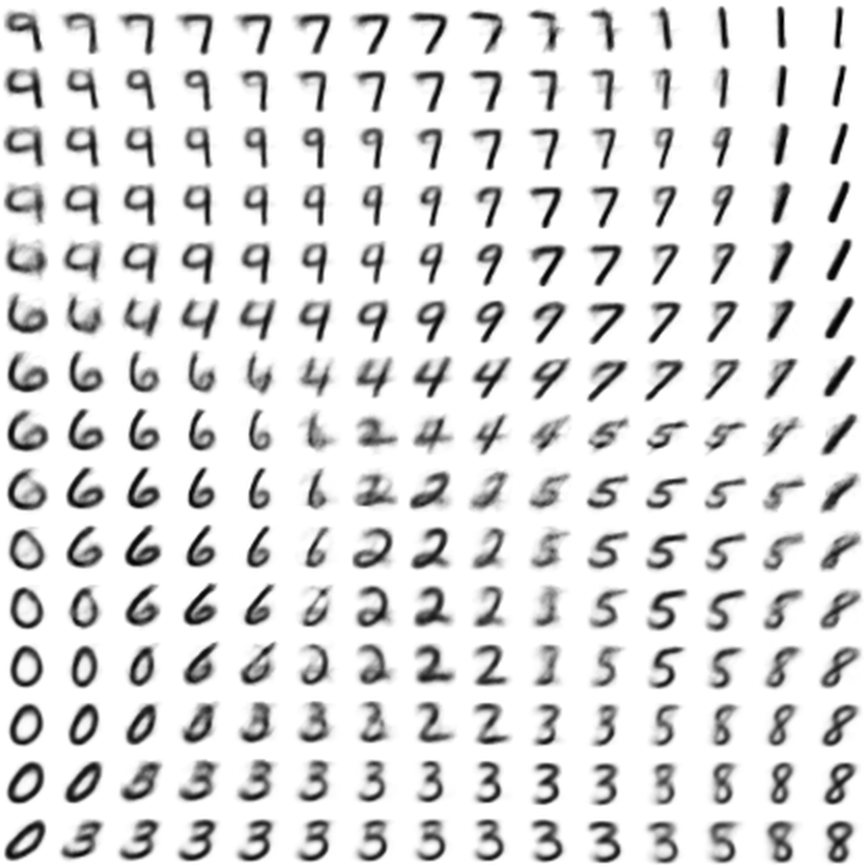


Figure 4. The decoder of the VAE with $k = 2$, running in your browser. Drag the point in the latent plane on the left, or focus it and use the arrow keys. The digit on the right is the decoder's output at that code. Grey dots are the encoder means of $2{,}000$ test digits, and the dashed circles have radius 1, 2 and 3 under the prior. Without JavaScript the figure shows the decoder applied to a 15×15 grid of codes placed at the quantiles of $\mathcal{N}(0, 1)$. *The online version of this figure is interactive.*

For this 2-dimensional model almost every region where the prior puts mass decodes into a digit, and neighbouring codes decode into similar digits. These are the two properties we asked for when we first turned the autoencoder into a generator. Transitions between classes are gradual, and a 7 becomes a 9 through images that look like both. Moving along a straight line between two codes gives a smooth sequence of images, which is called *interpolation* in the latent space.

Going further

Disentangled latent spaces

Directions in the latent plane also have a meaning. Along some of them the stroke tilts, and along others it thickens or one digit turns into another. A latent space is *disentangled* when each coordinate controls one such factor. The loss does not ask for this, and the prior cannot impose it, because $\mathcal{N}(0, I)$ does not change under rotations of the latent space. The diagonal covariance of the encoder breaks this symmetry and favours factors aligned with the axes (Rolinek, Zietlow, and Martius 2019). The β-VAE strengthens this pressure with $\beta > 1$, and reconstructions then get blurrier (Higgins et al. 2017). Without such built-in preferences, unsupervised disentanglement cannot be guaranteed (Locatello et al. 2019).

The prior we chose in advance need not suit the encoder we ended up with, so we should ask which prior would. The answer is the distribution of all the codes. Drawing an image from the data and then a code from its cloud gives a code with the density $\bar{q}_\phi(z) = \mathbb{E}_{X\sim\mathbb{P}^*}\, q_\phi(z \mid X)$, the *aggregate posterior*. To compare it with the prior we take the average KL term of the loss, called the *rate* (a name from coding theory that the fold-out below explains), and split it into two parts. One is the *mutual information* $I_q(x;z) = \mathbb{E}_X \,\mathrm{KL}\big(q_\phi(\cdot \mid X) \,\|\, \bar{q}_\phi\big)$, the average number of nats a code tells about its image, and the other compares the codes as a whole with the prior.

> **Proposition 15** (The rate splits in two)**.**
>
> $$\mathbb{E}_{X\sim\mathbb{P}^*}\,\mathrm{KL}\big(q_\phi(\cdot \mid X)\,\|\,p\big) = I_q(x;z) + \mathrm{KL}\big(\bar{q}_\phi \,\|\, p\big).$$

Proof.

Inside the KL, write $\log \frac{q_\phi(z|x)}{p(z)} = \log \frac{q_\phi(z|x)}{\bar{q}_\phi(z)} + \log \frac{\bar{q}_\phi(z)}{p(z)}$. Averaged over $X \sim \mathbb{P}^*$ and $Z \sim q_\phi(\cdot \mid X)$, the first part gives $I_q(x;z)$. The second part depends on z only, and the pair (X, Z) gives Z the distribution $\bar{q}_\phi$, so its average is $\mathbb{E}_{Z\sim\bar{q}_\phi}[\log \bar{q}_\phi(Z)/p(Z)] = \mathrm{KL}(\bar{q}_\phi \,\|\, p)$ (Hoffman and Johnson 2016). □

> **Corollary 16** (The best prior is the aggregate posterior)**.** *For a fixed encoder and decoder, the prior density p that maximizes $\mathbb{E}_{X\sim\mathbb{P}^*}[\mathcal{L}_{\theta,\phi}(X)]$ is $\bar{q}_\phi$.*

Proof.

By Proposition 9 the prior enters the average ELBO only through the rate, with a minus sign. By Proposition 15 the rate is $I_q(x;z) + \mathrm{KL}(\bar{q}_\phi \,\|\, p)$, and I_q does not depend on p. By Gibbs' inequality, $\mathrm{KL}(\bar{q}_\phi \,\|\, p) \geq 0$ with equality only at $p = \bar{q}_\phi$. □

The fixed prior $\mathcal{N}(0, I)$ is therefore the right prior only when the codes of the data fill it. Where $\bar{q}_\phi$ puts little mass and the prior puts a lot, prior codes land where the decoder has seen few training codes. This leads directly to the first failure below.

Going further

The rate-distortion reading of the ELBO

Write the loss as $-\mathcal{L} = \text{distortion} + \text{rate}$, with the *distortion* $-\mathbb{E}\log p_\theta(x \mid z)$ and the rate of Proposition 15, both in nats (Alemi et al. (2018) write them D and R). By Proposition 15 the rate bounds the information $I_q(x;z)$ in the code. Weighting the two terms as $\text{distortion} + \beta \cdot \text{rate}$ moves along a trade-off curve between a precise code and a cheap one, and at an optimum the slope of the curve is $-\beta$ (Alemi et al. 2018).

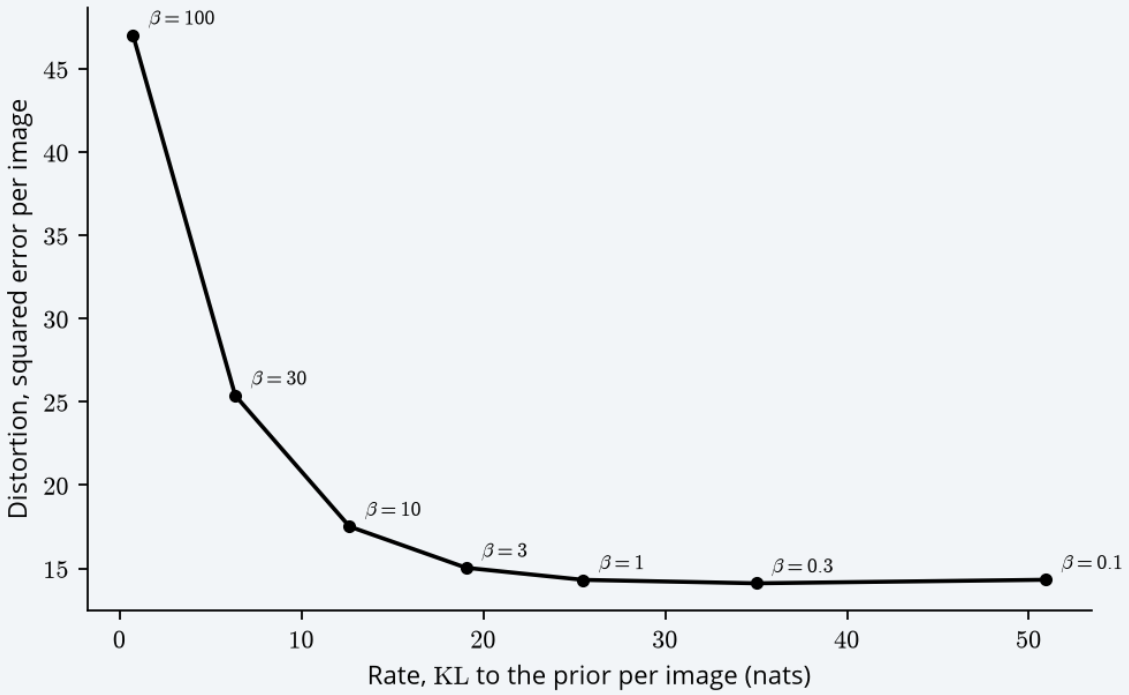


Figure 5. The β sweep on MNIST with $k = 8$ and 10 epochs per model. The distortion is plotted as the squared error per image, which equals $2\sigma_{\text{dec}}^2$ times the distortion, up to a constant.

As β falls from 100 to 0.1, the rate rises from 0.7 to 51 nats per image. The squared error falls from 47 to about 14 and then stops improving. Repeating $\beta = 0.1$, 0.3 and 1 with three seeds gives squared errors between 14.0 and 14.6 for all three, so lowering β below 1 adds rate without a measurable gain in squared error. In nats, the slope of the curve between $\beta = 3$ and $\beta = 1$ is -1.9, and between $\beta = 1$ and $\beta = 0.3$ it is -0.36. Each lies between the two values of $-\beta$, as the theory predicts. A latent coordinate counts as active when its encoder mean varies across test images with a variance above 0.01 (Burda, Grosse, and Salakhutdinov 2016). At $\beta = 100$ only 2 of the 8 coordinates stay active, and at $\beta = 30$ six of them do. Apart from the three repeated values, each β was trained once. These models train for 10 epochs against 30 for the main 8-dimensional VAE, so their squared errors are not comparable with its 12.7.

Going further

Tighter bounds

Averaging K importance weights before the logarithm gives the *importance weighted* bound $\mathcal{L}_K(x) = \mathbb{E}\big[\log \frac{1}{K} \sum_{i=1}^{K} \frac{p_\theta(x|z_i)\, p(z_i)}{q_\phi(z_i|x)}\big]$, where $z_1, \dots, z_K$ are drawn independently from the encoder (K counts samples, and k remains the latent dimension). The ELBO takes the logarithm of single weights and the log-likelihood is the logarithm of their expectation, and $\mathcal{L}_K$ lies between the two. It satisfies $\log p_\theta(x) \geq \mathcal{L}_{K+1} \geq \mathcal{L}_K$ with $\mathcal{L}_1$ the ELBO, and it approaches $\log p_\theta(x)$ when the weights are bounded (Burda, Grosse, and Salakhutdinov 2016, Theorem 1). The estimate of the gap on MNIST above is $\mathcal{L}_K$ with $K = 1{,}000$ minus the ELBO.

When it fails

We now have the formulas to explain what goes wrong. Each box below starts from what we see on trained models and traces it to what we have derived.

When it fails

Holes in the latent space

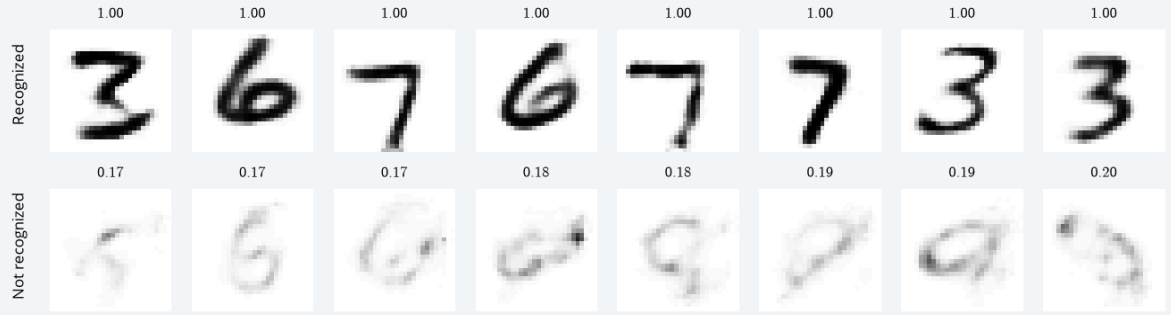


Figure 6. Prior samples of the 8-dimensional VAE with the classifier's confidence. The top row shows the most confident and the bottom row the least confident.

Of $1{,}000$ codes drawn from the prior, 53% decode into images that the shared classifier does not recognize with 90% confidence, and for 14% the confidence is below 50%. Prior codes typically lie at distance about $\sqrt{8} \approx 2.8$ from the origin, where $60{,}000$ clouds of standard deviation 0.05 fill a negligible share of 8-dimensional space. The aggregate posterior differs from the prior, which by Corollary 16 costs $\mathrm{KL}(\bar{q}_\phi \,\|\, p)$ nats per image, and prior codes land where the decoder has seen nothing. The clouds of the 2-dimensional model do fill the disk. Sampling codes from a second model fitted to the training codes replaces the prior by an estimate of $\bar{q}_\phi$.

When it fails

The code switches off

When the pull towards the prior weighs too much, or when the decoder is expressive enough to model images on its own (such as a decoder built like the autoregressive models of Chapter 1), the cheapest solution is $q_\phi(z \mid x) = p(z)$ for every image. The KL term then vanishes and the code carries no information about the image. This *posterior collapse* (Bowman et al. 2016) appears in the third column of the figure on the two terms of the loss, where 6 of the 8 coordinates are switched off.

The linear model gives the exact threshold. Its code uses a principal direction of the data only when the data vary along it more than the decoder noise, that is when the principal variance λ_j exceeds σ^2_{dec} (Proposition 17). By Proposition 11 a weight β on the KL term raises the threshold to $\beta\sigma^2_{\mathrm{dec}}$. Applied to the principal variances of MNIST, the rule roughly predicts when a nonlinear VAE starts to switch coordinates off. The test uses the VAEs of the rate-distortion box above, trained for 10 epochs at seven values of β from 0.1 to 100.

β	threshold $\beta\sigma_{\text{dec}}^2$	number of principal directions above it (of 784)	active coordinates predicted (of 8)	active coordinates measured (of 8)
10	0.3	35	8	8
30	0.9	13	8	6
100	3.0	3	3	2

The code has only 8 coordinates, so the prediction stops at 8. The largest principal variances of MNIST are 5.1, 3.7 and 3.3, out of a total of 52.7. Up to $\beta = 10$ all 8 coordinates stay active, as predicted. At $\beta = 30$ and 100 the nonlinear model trained for 10 epochs switches coordinates off a little earlier than the linear rule allows, at the same order of β. I expected the linear rule to say little about a nonlinear network trained for only 10 epochs. It gets the order of β right.

Going further

The collapse threshold of the linear model

> **Proposition 17** (When the linear model ignores its code)**.** *Take the linear model with $k = 1$, a fixed σ_{dec}^2 and b equal to the data mean. Let λ_1 be the largest eigenvalue of the data covariance $\Sigma_{\mathbb{P}^*}$, the variance of the data along its first principal direction ξ_1. The average log-likelihood is maximized by $a = \pm\sqrt{\lambda_1 - \sigma_{\text{dec}}^2}\,\xi_1$ if $\lambda_1 > \sigma_{\text{dec}}^2$, and by $a = 0$ otherwise. With $a = 0$ the posterior equals the prior for every image.*

Proof.

By Proposition 4 the model is $\mathcal{N}(b, C)$ with $C = aa^\top + \sigma_{\text{dec}}^2 I_d$. The average log-density of a Gaussian over data with mean b and covariance $\Sigma = \Sigma_{\mathbb{P}^*}$ is $-\frac{1}{2}\big(d\log 2\pi + \log\det C + \operatorname{tr}(C^{-1}\Sigma)\big)$, since $\mathbb{E}[(X-b)^\top C^{-1}(X-b)] = \operatorname{tr}\left(C^{-1}\,\mathbb{E}[(X-b)(X-b)^\top]\right)$.

Write $a = t\xi$ with $\|\xi\| = 1$ and $t \geq 0$, and $\lambda_C = \sigma_{\text{dec}}^2 + t^2$. The matrix C has the eigenvalue λ_C along ξ and σ_{dec}^2 on the $d-1$ orthogonal directions, so $\log\det C = \log\lambda_C + (d-1)\log\sigma_{\text{dec}}^2$. Its inverse is $C^{-1} = \big(I_d - (t^2/\lambda_C)\,\xi\xi^\top\big)/\sigma_{\text{dec}}^2$, as the product $(\sigma_{\text{dec}}^2 I_d + t^2\xi\xi^\top)(I_d - (t^2/\lambda_C)\,\xi\xi^\top) = \sigma_{\text{dec}}^2 I_d + \big(t^2 - (t^2/\lambda_C)(\sigma_{\text{dec}}^2 + t^2)\big)\xi\xi^\top = \sigma_{\text{dec}}^2 I_d$ shows. Hence

$$\operatorname{tr}(C^{-1}\Sigma) = \frac{1}{\sigma_{\text{dec}}^2}\Big(\operatorname{tr}\Sigma - \frac{t^2}{\lambda_C}\,\xi^\top\Sigma\xi\Big).$$

For a fixed t only this term depends on ξ, and it is smallest when $\xi^\top\Sigma\xi$ is largest, at $\xi = \pm\xi_1$ with $\xi_1^\top\Sigma\xi_1 = \lambda_1$. Substituting $t^2 = \lambda_C - \sigma_{\text{dec}}^2$ and dropping the terms that depend on neither t nor ξ, maximizing the log-likelihood means minimizing over $\lambda_C \geq \sigma_{\text{dec}}^2$

$$\log \lambda_C - \frac{(\lambda_C - \sigma_{\text{dec}}^2)\,\lambda_1}{\sigma_{\text{dec}}^2\,\lambda_C} = \log \lambda_C + \frac{\lambda_1}{\lambda_C} - \frac{\lambda_1}{\sigma_{\text{dec}}^2}.$$

The derivative of $\log \lambda_C + \lambda_1/\lambda_C$ is $(\lambda_C - \lambda_1)/\lambda_C^2$. If $\lambda_1 > \sigma_{\text{dec}}^2$ the minimum is at $\lambda_C = \lambda_1$, so $t^2 = \lambda_1 - \sigma_{\text{dec}}^2$. Otherwise the function increases on $[\sigma_{\text{dec}}^2, \infty)$, the minimum is at $\lambda_C = \sigma_{\text{dec}}^2$, and $t = 0$. With $a = 0$, Proposition 4 gives $M = \sigma_{\text{dec}}^2$ and the posterior $\mathcal{N}(0, 1)$, which is the prior. □

For $k = 1$ the posterior is a one-dimensional Gaussian, so a Gaussian encoder can equal it and the maximum of the ELBO is this maximum of the likelihood. For general k the stationary points of the likelihood of probabilistic PCA give each column of A a principal direction and a squared length $\lambda_j - \sigma_{\text{dec}}^2$, or the length 0, so a direction with $\lambda_j \le \sigma_{\text{dec}}^2$ can only be switched off (Tipping and Bishop 1999, Appendix A). The zero columns are the posterior collapse of the linear model (Lucas et al. 2019, sec. 4.1).

When it fails

Mass between the modes

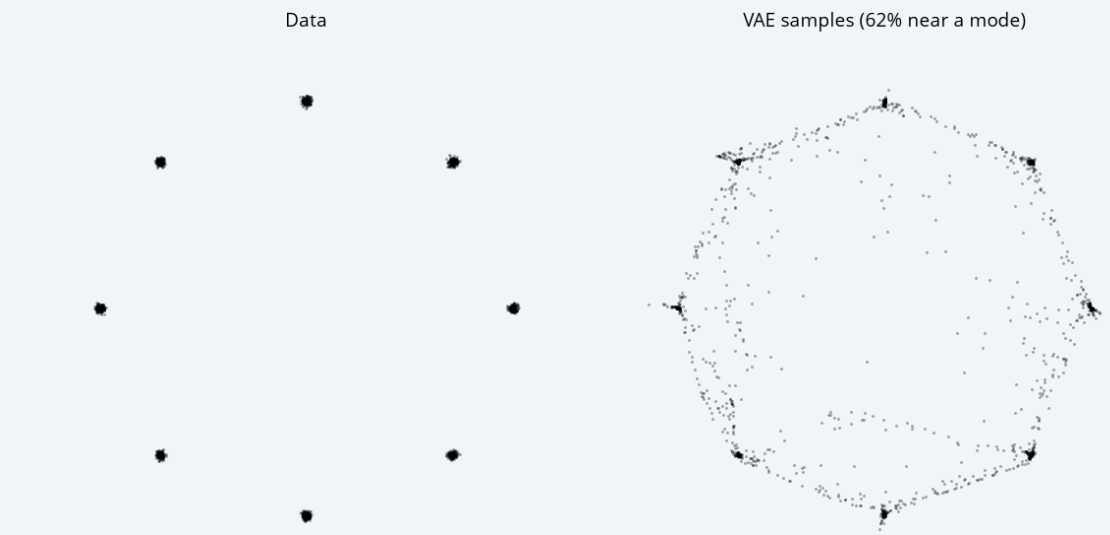


Figure 7. On the left, $2{,}000$ points of the ring of 8 Gaussians. On the right, $2{,}000$ of the $5{,}000$ samples of a VAE trained on it, shown as decoder means. The data and the code are 2-dimensional and $\sigma_{\text{dec}}^2 = 0.001$.

The VAE finds all 8 modes, but only 61.5% of its samples land within three standard deviations of a mode, against 99.1% for real data. Some bridge between the modes must exist, because a continuous decoder fed with a connected prior has a connected image. Prior codes that fall between the clusters of training codes decode onto the bridges. The forward KL of Chapter 2 averages over the data, so model mass where there are no data costs nothing directly and only leaves less mass for the modes. The decoder noise ($\sigma_{\text{dec}} \approx 0.03$) is also wider than the modes (0.02), so the noisy model cannot resolve them exactly.

Where the blur comes from

Chapter 1 listed blurry samples as the cost of the VAE, and the latent explorer shows where they start. The codes between the cloud of the 7s and the cloud of the 9s decode into images that look like both. Several training images share each of these codes, and the decoder must answer one image for all of them. With the squared error of the Gaussian decoder, its best answer is their average.

> **Proposition 18** (The best decoder is a conditional mean). *Fix the encoder, draw $X \sim \mathbb{P}^*$ and $Z \sim q_\phi(\cdot \mid X)$, and assume $\mathbb{E}\|X\|^2 < \infty$. Among all functions $g : \mathbb{R}^k \to \mathbb{R}^d$, the expected squared error $\mathbb{E}\|X - g(Z)\|^2$ is minimized by the conditional mean $g^*(z) = \mathbb{E}[X \mid Z = z]$. The minimum is $\mathbb{E}\|X - \mathbb{E}[X \mid Z]\|^2$, the average conditional variance of the images that share a code.*

Proof.

Write $g^*(Z) = \mathbb{E}[X \mid Z]$ and expand around it,

$$\mathbb{E}\|X - g(Z)\|^2 = \mathbb{E}\|X - g^*(Z)\|^2 + 2\,\mathbb{E}\big[(X - g^*(Z))^\top (g^*(Z) - g(Z))\big] + \mathbb{E}\|g^*(Z) - g(Z)\|^2.$$

In the middle term, condition on Z. The vector $g^*(Z) - g(Z)$ is then fixed and $\mathbb{E}[X - g^*(Z) \mid Z] = 0$, so the middle term is 0 by the tower property. The last term is non-negative and vanishes for $g = g^*$, which gives both claims. □

The conditional mean is an explicit average of images. A code z is reached by the clouds of many images, and

$$\mathbb{E}[X \mid Z = z] = \frac{\mathbb{E}_{X \sim \mathbb{P}^*}\big[X\, q_\phi(z \mid X)\big]}{\bar{q}_\phi(z)},$$

the average of the images weighted by how strongly their clouds reach z, divided by the aggregate posterior of Proposition 15.

So blur comes from averaging, and averaging needs several images to share a code. Two things make them share one. An 8-number code cannot tell apart images that differ only in details it has no room for, and the KL penalty makes the encoder clouds overlap. Noise on the displayed images plays no part, since we show decoder means. The decoder variance enters only through the weight of the KL term.

We can measure how much of the blur the KL term causes. We score blur by the share of an image's spectral power above a quarter cycle per pixel, where strokes have their edges. It is 10.5% for real test digits and 4.7% for the reconstructions of our VAE. I expected halving the KL weight to make the reconstructions sharper, since the clouds would overlap less. The VAE trained with the best decoder variance, which weighs the KL term about half as much, scores 4.9%, so the share barely moved. The plain autoencoder of the same size has no KL term and no

noise on its code. It scores 4.4%, so its reconstructions are no sharper. It was trained for only 20 epochs against 30 for the VAEs, so this comparison is not clean. Only a much heavier penalty changes the picture. With the KL term weighted 100 times more, the share falls to 0.15%. Each of these models was trained once, so the small differences between them may be noise.

So at our settings the KL term does not look like the main cause of the blur. That leaves the 8-number code, whose length we did not vary, and the squared-error loss.

This measure has a limit. A higher share does not always mean a better image, because artefacts carry high frequencies too. Prior samples score higher than reconstructions, 5.6% against 4.7% for our VAE, and the prior samples of the plain autoencoder score 4.9% although almost none of them are digits.

Diffusion models (Chapter 7) are built on the fact behind this section, that a regression predicts a conditional average.

Going further

Extensions

A *conditional VAE* (Sohn, Lee, and Yan 2015) lets the encoder and the decoder see the label. We then choose the digit, and the code describes its style. A *VQ-VAE* (Oord, Vinyals, and Kavukcuoglu 2017) uses discrete codes. Its encoder sends an image to the nearest vector of a learned dictionary, as k-means does. The k-means algorithm is itself an autoencoder whose code is the index of a cluster and whose decoder returns the cluster's centre. A second model then learns to sample the indices, and the samples are less blurry. *Latent diffusion* (Rombach et al. 2022), the subject of Chapter 9, compresses images with an autoencoder that has a light KL penalty and generates in its latent space with a diffusion model.

Check yourself

1. Why not maximize $\log p_\theta(x)$ directly?

   Answer

   > It is an integral of a neural network over the whole latent space with no closed form, and estimating it by sampling the prior needs a number of codes that grows geometrically with k (Proposition 5). The ELBO replaces it by an expectation under the encoder that we can estimate and differentiate from single codes.

2. What is the ELBO when $q_\phi(z \mid x)$ is the prior for every x?

   Answer

   > The KL term is 0, so the ELBO is $\mathbb{E}_{z\sim p(z)}[\log p_\theta(x \mid z)]$, the lower bound of Proposition 3. The code carries no information about x, and the best decoder maps every code to the mean image. This is posterior collapse.

3. Doubling σ^2_{dec} has the same minimizers as which β?

   Answer

> $\beta = 2$, by Proposition 11. The loss $\frac{1}{2 \cdot 2\sigma_{\text{dec}}^2}\|x - g(z)\|^2 + \mathrm{KL}$ is half of $\frac{1}{2\sigma_{\text{dec}}^2}\|x - g(z)\|^2 + 2\,\mathrm{KL}$.

4. A linear VAE with $k = 1$ and $\sigma_{\text{dec}}^2 = 0.03$ is trained on MNIST, whose largest principal variance is 5.1. From which weight β on the KL term does its code switch off?

   Answer

   > By Proposition 17 and Proposition 11 the code is used while $5.1 > 0.03\,\beta$, so it switches off for $\beta \geq 5.1/0.03$, about 170.

5. A VAE is trained with the prior $\mathcal{N}(0, I)$. Which prior would give it a higher average ELBO without retraining?

   Answer

   > The aggregate posterior $\bar{q}_\phi$, the distribution of the codes of the training images (Corollary 16). The gain is $\mathrm{KL}(\bar{q}_\phi \,\|\, \mathcal{N}(0, I))$ nats per image.

6. The importance weights of the encoder vary by a factor of hundreds on MNIST. Why is one code per image still enough to train?

   Answer

   > The loss uses the logarithm of the weight, whose standard deviation is 6.7 nats. The single-code estimate is unbiased for the ELBO, and its noise averages out over the minibatch and the steps.

Summary

We began by hoping that the decoder of an autoencoder would generate, and with 0.3% of its prior samples recognized as digits it did not, because nothing controlled where its codes lay. Replacing each code by a Gaussian cloud and pulling the clouds towards the prior raised the share to 47%. At that stage these were two repairs with no theory behind them.

The theory came from asking for a likelihood. Gaussian noise on the decoder gives the generator a density, but one that is an integral over codes. Sampling that integral from the prior fails badly, and sampling it from the encoder works better, although the weights still vary a lot and the estimate of the likelihood stays poor. Taking the logarithm before averaging gives the evidence lower bound, which falls short of the log-likelihood by the KL from the encoder to the posterior. Averaged over the data, the negative bound equals $\mathrm{KL}(\mathbb{P}^* \,\|\, \mathbb{P}_\theta)$ plus that gap, up to a constant, so it is an upper bound on the distance. The gap is at least about 14 nats for the median MNIST digit. Split in two, the bound gives back our two repairs, and the decoder variance sets their balance. The reparameterization trick then trains it from one code per image.

The failures can be read off the same formulas. The codes of the data do not fill the prior, whose best replacement is the aggregate posterior. In the linear model the code switches off along directions whose data variance is below $\beta\sigma_{\text{dec}}^2$, and the nonlinear VAE follows this rule roughly. The squared-error decoder outputs the average of the images that share a code, which makes the samples blurry. Halving the KL weight or removing it did not sharpen the reconstructions, so at our settings it does not look like the main cause.

Chapter 4 trains a generator without a likelihood. A generative adversarial network trains the generator against a second network that learns to tell real images from generated ones. It needs no density and no decoder noise, and its samples are sharp. Its surrogate is a lower bound, and Chapter 2's proposition on surrogates showed that training can exploit the gap of a lower bound.

Alemi, Alexander A., Ben Poole, Ian Fischer, Joshua V. Dillon, Rif A. Saurous, and Kevin Murphy. 2018. "Fixing a Broken ELBO." In *International Conference on Machine Learning (ICML)*.

Baldi, Pierre, and Kurt Hornik. 1989. "Neural Networks and Principal Component Analysis: Learning from Examples Without Local Minima." *Neural Networks* 2 (1): 53–58.

Bowman, Samuel R., Luke Vilnis, Oriol Vinyals, Andrew M. Dai, Rafal Jozefowicz, and Samy Bengio. 2016. "Generating Sentences from a Continuous Space." In *Conference on Computational Natural Language Learning (CoNLL)*.

Burda, Yuri, Roger Grosse, and Ruslan Salakhutdinov. 2016. "Importance Weighted Autoencoders." In *International Conference on Learning Representations (ICLR)*.

Cremer, Chris, Xuechen Li, and David Duvenaud. 2018. "Inference Suboptimality in Variational Autoencoders." In *International Conference on Machine Learning (ICML)*.

Higgins, Irina, Loic Matthey, Arka Pal, Christopher Burgess, Xavier Glorot, Matthew Botvinick, Shakir Mohamed, and Alexander Lerchner. 2017. "Beta-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework." In *International Conference on Learning Representations (ICLR)*.

Hoffman, Matthew D., and Matthew J. Johnson. 2016. "ELBO Surgery: Yet Another Way to Carve up the Variational Evidence Lower Bound." In *NIPS 2016 Workshop on Advances in Approximate Bayesian Inference*.

Kingma, Diederik P., Tim Salimans, Rafal Jozefowicz, Xi Chen, Ilya Sutskever, and Max Welling. 2016. "Improving Variational Inference with Inverse Autoregressive Flow." In *Advances in Neural Information Processing Systems (NeurIPS)*.

Kingma, Diederik P., and Max Welling. 2014. "Auto-Encoding Variational Bayes." In *International Conference on Learning Representations (ICLR)*.

Loaiza-Ganem, Gabriel, and John P. Cunningham. 2019. "The Continuous Bernoulli: Fixing a Pervasive Error in Variational Autoencoders." In *Advances in Neural Information Processing Systems (NeurIPS)*.

Locatello, Francesco, Stefan Bauer, Mario Lucic, Gunnar Rätsch, Sylvain Gelly, Bernhard Schölkopf, and Olivier Bachem. 2019. "Challenging Common Assumptions in the Unsupervised Learning of Disentangled Representations." In *International Conference on Machine Learning (ICML)*.

Lucas, James, George Tucker, Roger Grosse, and Mohammad Norouzi. 2019. "Don't Blame the ELBO! A Linear VAE Perspective on Posterior Collapse." In *Advances in Neural Information Processing Systems (NeurIPS)*.

Mattei, Pierre-Alexandre, and Jes Frellsen. 2018. "Leveraging the Exact Likelihood of Deep Latent Variable Models." In *Advances in Neural Information Processing Systems (NeurIPS)*.

Oord, Aaron van den, Oriol Vinyals, and Koray Kavukcuoglu. 2017. "Neural Discrete Representation Learning." In *Advances in Neural Information Processing Systems (NeurIPS)*.

Pope, Phillip, Chen Zhu, Ahmed Abdelkader, Micah Goldblum, and Tom Goldstein. 2021. "The Intrinsic Dimension of Images and Its Impact on Learning." In *International Conference on Learning Representations (ICLR)*.

Rezende, Danilo Jimenez, Shakir Mohamed, and Daan Wierstra. 2014. "Stochastic Backpropagation and Approximate Inference in Deep Generative Models." In *International Conference on Machine Learning (ICML)*.

Rolinek, Michal, Dominik Zietlow, and Georg Martius. 2019. "Variational Autoencoders Pursue PCA Directions (by Accident)." In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

Rombach, Robin, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. 2022. "High-Resolution Image Synthesis with Latent Diffusion Models." In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

Rybkin, Oleh, Kostas Daniilidis, and Sergey Levine. 2021. "Simple and Effective VAE Training with Calibrated Decoders." In *International Conference on Machine Learning (ICML)*.

Sohn, Kihyuk, Honglak Lee, and Xinchen Yan. 2015. “Learning Structured Output Representation Using Deep Conditional Generative Models.” In *Advances in Neural Information Processing Systems (NeurIPS)*.

Tipping, Michael E., and Christopher M. Bishop. 1999. “Probabilistic Principal Component Analysis.” *Journal of the Royal Statistical Society: Series B (Statistical Methodology)* 61 (3): 611–22. https://doi.org/10.1111/1467-9868.00196.

❝ Cite this chapter

Bouftini, I. (2026). Variational autoencoders. Chapter 3 of the course Generative Models for Images. https://ibouftini.github.io/courses/generative-models/variational-autoencoders/

```
@misc{bouftini2026variational,
  author = {Bouftini, Imade},
  title  = {Variational autoencoders},
  year   = {2026},
  note   = {Chapter 3 of the course Generative Models for Images},
  url    = {https://ibouftini.github.io/courses/generative-models/variational-
autoencoders/}
}
```

Printed from https://ibouftini.github.io/courses/generative-models/variational-autoencoders/. The online version has the interactive figures and any later corrections.

© 2026 Imade Bouftini GitHub LinkedIn